Ce qu'EFE sait faire aujourd'hui — guide explicatif complet

Document pédagogique pour comprendre **précisément** ce que la plateforme EFE est capable d'exécuter à ce jour (24 mai 2026). Adressé à quelqu'un qui découvre le projet, à un investisseur en due diligence technique, ou à un ingénieur qui doit décider quoi consommer. **Méthode** : analyse directe du code (47 crates Rust, 187 fichiers Python), du CHANGELOG (8 mois de sprints documentés), des tests (146 fichiers de tests), des handlers HTTP, des 78 tools MCP, des architectures M5–M50 et des audits internes. Distingue strictement le **shippé/testé**, le **stubbé/interface stable**, et le **planifié/non commencé**.

---

0. Comment lire ce document

Chaque section décrit **une capacité fonctionnelle** d'EFE. Pour chaque capacité, vous trouverez :

- **À quoi ça sert** — le besoin métier
- **Comment ça marche** — l'idée centrale, sans jargon
- **Ce qui est shippé** — preuve dans le code
- **Comment l'utiliser** — endpoint, tool MCP, ou exemple concret
- **Limites actuelles** — ce qui n'est pas encore là

Légende :

- **[SHIPPED]** = code shippé, testé, en CI
- **[STUB]** = code shippé mais avec stub pour la dépendance externe (à wirer en prod)
- **[N/A]** = pas shippé, planifié
- **[S&S]** = extension Scout & Synthesize, non shippée dans EFE actuel

---

1. Le modèle mental EFE en 5 minutes

1.1 Le problème qu'EFE résout

Un agent IA en production fait souvent **cent appels LLM en série** pour traiter un document : extraction du JSON, validation du format, classification de l'intent, résumé, vérification de cohérence… Cette cascade séquentielle a quatre pathologies graves :

1. **Latence** : neuf minutes pour un workflow qui devrait prendre dix secondes
2. **Coût** : 50 cents × 100 appels × 1000 workflows/jour = des centaines de milliers d'euros par mois
3. **Hallucinations sur tâches déterministes** : un LLM se trompe 5–15 % du temps quand on lui demande de parser un JSON ou d'extraire une date — taux inacceptable en pharma, finance, défense
4. **Audit opaque** : aucun régulateur ne peut rejouer ni vérifier les décisions prises

EFE s'attaque à ces quatre pathologies *simultanément*.

1.2 L'approche : remplacer la cascade par un essaim parallèle d'agents schema-validés

Au lieu d'enchaîner 100 LLM, EFE exécute **100 nano-agents en parallèle**, chacun spécialisé sur une sous-tâche, chacun avec un **schéma d'entrée et de sortie figé**, chacun produisant une **ligne d'audit cryptographique**.

Trois familles de nano-agents coexistent :

```
┌──────────────────────────────────────────────────────────────┐
|  Mode A : Rust pur, 1-10 ms, déterminisme mathématique       |
|  Ex: json_validate, regex_extract, date_normalize            |
|  Coût ≈ 0 €. Audit triviale (input -> output bit-identique).  |
├──────────────────────────────────────────────────────────────┤
|  Mode B : ML local en microVM, 50-200 ms, déterminisme seeded |
|  Ex: ner_clinical, ner_legal, intent_classify, embed_text    |
|  Coût ≈ 0.1 ¢. Avec seed fixé, output bit-identique.         |
├──────────────────────────────────────────────────────────────┤
|  Mode C : LLM + JSON schema strict, 500-2000 ms, replayable  |
|  Ex: summarize_strict, extract_structured, qa_grounded       |
|  Coût ≈ 1-5 ¢. Cache TTL garantit replay dans la fenêtre.    |
└──────────────────────────────────────────────────────────────┘
```

L'observation clé est qu'une charge utilisateur réelle se répartit *typiquement* en **70-80 % Mode A, 15-25 % Mode B, 3-10 % Mode C**. Les appels les plus chers sont aussi les plus rares. C'est ce qui rend l'économie viable.

1.3 Les quatre couches architecturales

EFE est organisé en quatre couches logiques, chacune apportant une valeur indépendante :

Couche	Rôle	Pourquoi c'est différenciant
1. Inférence latent-space	Calcul tensoriel de consensus multi-agent	Coût marginal d'un agent supplémentaire en microsecondes vs secondes pour un LLM
2. Meta-swarm A1	Fédération entre instances distinctes	Permet le multi-région avec souveraineté des données
3. Audit transverse	HMAC chains + Merkle epochs + replay engine	**Non-retrofittable** : un concurrent qui n'a pas été conçu autour de l'audit dès le départ ne peut pas l'ajouter rétroactivement
4. Nano-agent platform	Exécution + orchestration + marketplace de publishers	Surface d'API pour les agents Claude/GPT/custom via MCP

1.4 Pourquoi cette architecture, pas une autre

Trois principes guident chaque décision de design :

1. **Audit-by-construction** — aucune opération ne peut contourner la chaîne d'audit. Garantie par le système de types Rust : tout `Runtime` reçoit un `CallContext` qui contient l'`AuditWriter`. Pas d'opt-out.
1. **Schema-validated I/O** — chaque nano-agent a un JSON Schema d'entrée et de sortie versionnés. Une PR qui change un schéma sans bumper la version est rejetée par la CI.
1. **Tier-based publisher trust** — un agent publié par un tiers monte en confiance par paliers cryptographiquement vérifiables (Tier 0 -> Tier Audit). Pas de "blind trust".

---

2. Vue d'ensemble en chiffres (état au 24 mai 2026)

Catégorie	Valeur
Crates Rust en workspace	**47**
Lignes de code Rust	**~50 000**
Fichiers de tests	**146** dans `crates/`
Tests passants	**> 5 000** (selon investor brief, audit non re-runné depuis purge `/tmp`)

Catégorie	Valeur
MCP tools Python exposés	**78** (inventaire alphabétique vérifié)
Endpoints HTTP Gateway	**21** (4 publics + 17 authentifiés)
Sous-commandes CLI `efe-admin`	**14**
Workflows GitHub Actions	**89**
Helm charts	**4** (`efe-stack`, `efe-agent-runner`, `efe-meta-swarm-loadgen`, `efe-nano-firecracker`)
Architecture docs principales	**24+** (M5-M12, M21-M33, A1 ×2 phases, M50, Letter A-E, MODE1)
SLOs publiés (core, en CI)	**5** ; consolidation master = **18** SLOs
Audit reports de sprints	**18** dans `docs/audits/` (D6 à D18)
Version GA atteinte	**v1.0.0 le 9 mai 2026** (release pipeline R-100)
Dernier sprint	**v0.9.0 le 12 mai 2026** (Φ-D-OPS — unification opérationnelle)
Durée totale de construction	Sprint 0 le 15 sept 2025 -> aujourd'hui = **~8 mois disciplinés**

3. Capacité — Classifier des documents (le cas d'usage le plus mature)

3.1 À quoi ça sert

Un client donne à EFE un corpus de textes (tickets de support, documents juridiques, fiches cliniques, articles…) et une **taxonomie** (liste de labels). EFE retourne pour chaque texte le label correspondant avec un score de confiance, en *millisecondes* et avec une trace d'audit complète.

3.2 Comment ça marche : trois modes de classification distincts

EFE propose **trois familles** de classification correspondant à trois compromis coût/qualité :

Mode 1 — Embeddings + ancres centroides (le moins cher, le plus rapide)

```
Texte d'entrée
    ↓
[Embedder local (sentence-transformers ou InBedder)]
    ↓
Vecteur 384-dim ou 1024-dim
    ↓
[Calcul de distance à chaque ancre centroide du catalogue]
    ↓
Label = argmin(distance) si confiance > seuil, sinon "uncertain"
```

- **Latence p95** : 50 ms (small), 200 ms (medium), 1 s (large) — SLO en CI
- **Coût** : ≈ 0 € (pas d'appel LLM)
- **Déterminisme** : mathématique strict
- **Sweet spot** : taxonomies de 10 à 200 labels, exemples par label fournis ou générés

Mode 2 — LLM-routed avec budget + cache + circuit breaker

Quand Mode 1 retourne "uncertain" (faible confiance), Mode 2 escalade vers un LLM :

```
Mode 1 -> confidence < seuil
    ↓
[efe-llm-router]
```

```
    ↓
- Sélection du provider selon budget tenant
- Cache Redis lookup (BLAKE3 hash du prompt)
- Si miss : appel Anthropic/OpenAI avec JSON schema strict
- Validation de la sortie + retry si invalide
- Provenance per-call enregistrée
    ↓
Label avec confiance LLM
```

- **5 tools MCP** : `efe_classify_mode2`, `efe_get_llm_budget`, `efe_set_llm_budget`, `efe_invalidate_llm_cache`, `efe_list_llm_providers`
- **Architecture** : `docs/EFE_M21_MODE2_LLM_ROUTED_ANCHORS_ARCHITECTURE.md`
- **Cache TTL** : 5 minutes par défaut, configurable

Mode 3 — Multi-tour avec human-in-the-loop + concept stack

Pour les classifications complexes où plusieurs allers-retours sont nécessaires :

```
Texte -> première proposition LLM
    ↓
[Concept stack] (mémoire de travail conceptuelle)
    ↓
[Human reviewer] approuve / rejette / amende
    ↓
Mise à jour du concept stack
    ↓
Itération jusqu'à convergence
```

- **5 tools MCP** : `efe_classify_mode3`, `efe_approve_mode3_review`, `efe_reject_mode3_review`, `efe_get_mode3_reviews`, `efe_get_concept_stack`
- **Architecture** : `docs/EFE_M22_MODE3_*_ARCHITECTURE.md`

3.3 Ce qui est shippé et testable immédiatement

Le catalogue `support_triage` (4 intents : fraud / bug / question / billing) est livré clé en main. Le test minimal pour un nouveau développeur :

```
# examples/01_quickstart_classify.py — version simplifiée
from efe_tool import AnchorCatalog, EfeClient, LocalSTEmbedder

embedder = LocalSTEmbedder.from_env()
catalog = AnchorCatalog.load("catalogs/support_triage.yaml", embedder=embedder)

with EfeClient(endpoint="127.0.0.1:50051", embedder=embedder) as client:
    for text in ["Someone charged my card without consent.",
                 "L'application plante quand j'ouvre les paramètres."]:
        r = client.classify(text, catalog)
        print(f"{text[:40]} -> {r.label} ({r.confidence:.2f})")
```

Sortie attendue :

```
Someone charged my card without conse... -> fraud      (0.92)
L'application plante quand j'ouvre les... -> bug       (0.91)
```

3.4 Examples Python shippés (5 + 2 tutoriels)

Fichier	Démontre
`01_quickstart_classify.py`	First classification en 30 secondes
`02_custom_catalog.py`	Définir ses propres intents en YAML
`03_multilang_streaming.py`	Streaming + multilingue (FR/EN/DE/ES)
`04_production_client.py`	Pattern production : retry, timeouts, logging
`05_catalog_admin.py`	Gestion programmatique des catalogues

Fichier	Démontre
hybrid_classification.py	Mode 1 + Mode 2 combinés (escalade)
tutorial-1-classify-text/	Tutoriel pas à pas (texte)
tutorial-2-batch-classify/	Tutoriel batch processing

3.5 Tools MCP pour classification (12)

efe_classify, efe_bulk_classify, efe_list_catalogs, efe_register_catalog, efe_evolve_catalog, efe_curate_catalog, efe_detect_drift, efe_diagnose_catalog, efe_explain_classification, efe_explain_misclassification, efe_score_distribution, efe_compare_classes.

---

4. Capacité — Les 20 nano-agents starter pack

4.1 Mode A — 10 agents Rust pur, déterminisme mathématique

Ces agents sont **garantis bit-identiques** pour les mêmes entrées, forever. Pas de modèle, pas d'aléa.

Agent	Input	Output	Use case régulé
json_validate	{value, schema}	{valid: bool, errors: [...]}	Validation conformité FINMA/AI Act
xml_validate	{xml, xsd}	{valid: bool, ...}	Validation déclarations DORA
regex_extract	{text, pattern, group}	{matches: [{start, end, value}]}	Extraction numéros de RCS, SIREN
date_normalize	{date_str, source_locale}	{iso8601: "YYYY-MM-DDTHH:MM:SSZ"}	Normalisation pour audit logs
currency_normalize	{amount_str, locale}	{value_cents: int, currency: ISO4217}	Pré-traitement KYC AML
csv_normalize	{csv, expected_columns}	{rows: [...], errors: [...]}	Pré-traitement données reg.
markdown_extract_headings	{markdown}	{headings: [{level, text, anchor}]}	Index documents internes
html_extract_main_content	{html}	{title, body_text, lang}	Pré-traitement crawls
pdf_extract_text	{pdf_bytes}	{pages: [{text, bbox}]}	Pré-traitement essais cliniques
sql_parse_safe	{sql}	{ast: {...}, errors: []}	Validation reqs analystes

Pourquoi Rust pur et pas LLM ? Parce que pour ces tâches, un LLM hallucine 5-15 % du temps et coûte 1-5 cents par appel — alors qu'un parseur déterministe coûte ≈ 0 et ne se trompe jamais. C'est **l'aberration économique** qu'EFE corrige en priorité.

4.2 Mode B — 5 agents ML local, déterminisme seeded

Ces agents tournent dans une **microVM Firecracker isolée** (en prod) ou un stub in-process (en dev). Modèles pré-chargés en mémoire au boot.

Agent	Modèle interne	Use case
ner_clinical	NER médical fine-tuné (SciSpaCy-like)	Extraction médicaments, dosages, phases d'essai (pharma)

Agent	Modèle interne	Use case
`ner_legal`	NER légal fine-tuné	Extraction parties, dates clés, montants (contrats, jugements)
`intent_classify_generic`	Encoder + classifier head	Détection d'intention multi-classe
`lang_detect`	fastText langid	Détection 100+ langues, < 5 ms
`embed_text`	bge-small ou bge-m3 (selon config)	Embeddings pour retrieval

Garantie déterministe : avec un seed fixé dans la config, deux invocations sur la même entrée produisent **strictement le même output** (testé via `phase_2_mode_b.rs`).

4.3 Mode C — 5 agents LLM augmentés, déterminisme cache-replayable

Ces agents appellent un LLM avec un **JSON Schema strict** sur la sortie. Si la sortie ne valide pas, retry jusqu'à 3 fois avec feedback. Cache Redis 5 min par défaut -> replay garanti dans la fenêtre.

Agent	LLM appelé	JSON Schema de sortie
`summarize_strict`	Claude / GPT	`{summary: str, sentence_count: int}`
`extract_structured`	Claude / GPT	Schema fourni par caller
`classify_topic_llm`	Claude / GPT	`{topic: enum, confidence: 0..1}`
`sentiment_nuanced`	Claude / GPT	`{polarity, intensity, dimensions: {...}}`
`qa_grounded`	Claude / GPT	`{answer: str, sources_used: [int]}`

Pourquoi un LLM pour ces tâches ? Parce que c'est exactement le sweet spot d'un LLM : tâches sémantiques où l'ambiguïté humaine est légitime. Mais on **contraint la sortie** par schéma, donc l'output reste structuré et audit-grade.

4.4 Les 6 SwarmAgents de pré-analyse (différents du starter pack)

Pour l'orchestration cyclique via blackboard, EFE expose un trait `SwarmAgent` distinct :

Agent	Rôle
`ner_extractor`	Wrapper SwarmAgent autour de la NER
`sentiment_classifier`	Sentiment via blackboard
`language_detector`	Lang detect via blackboard
`pii_redactor`	PII redaction avant LLM
`summary_writer`	Synthèse multi-agent
`schema_validator`	Validation finale du document

La différence : un `NanoAgent` est sans état (input -> output via dispatcher), un `SwarmAgent` lit/écrit dans le blackboard partagé. Les deux peuvent se ponter via `efe-nano-swarm-adapter`.

---

5. Capacité — Orchestrer un DAG d'opérations

5.1 À quoi ça sert

Au lieu d'écrire un script séquentiel "appel A -> appel B -> appel C", EFE permet de déclarer un **graphe de dépendances** et il s'occupe d'exécuter en parallèle ce qui peut l'être.

5.2 Exemple concret de DAG

Un workflow "traiter un contrat juridique" :

```
                                        ┌── ner_legal ──────────┐
                                        |   (extract parties)   |
PDF brut -> pdf_extract_text ── html_clean ── ner_clinical ── aggregate
                                        |   (if mention drugs)  |
                                        └── sentiment_nuanced ──┘
                                            (Mode C, parallèle)
```

Format `SwarmRequest` (extrait JSON simplifié) :

```
{
  "request_id": "01HF...",
  "tenant_id": "acme-pharma",
  "operations": [
    {"op_id": "extract",  "agent": "pdf_extract_text",  "input": {...}, "depends_on": []},
    {"op_id": "clean",    "agent": "html_extract_main", "input": {"from": "extract"}, "depends_on": ["extract"]},
    {"op_id": "ner_l",    "agent": "ner_legal",         "input": {"from": "clean"},   "depends_on": ["clean"]},
    {"op_id": "ner_c",    "agent": "ner_clinical",      "input": {"from": "clean"},   "depends_on": ["clean"]},
    {"op_id": "sentiment","agent": "sentiment_nuanced", "input": {"from": "clean"},   "depends_on": ["clean"]},
    {"op_id": "final",    "agent": "aggregate",         "input": {"from": ["ner_l","ner_c","sentiment"]},
                                                        "depends_on": ["ner_l","ner_c","sentiment"]}
  ],
  "max_parallelism": 8,
  "timeout_ms": 30000,
  "cost_cap_cents": 50
}
```

Le dispatcher :

1. Trie topologiquement -> `[extract] -> [clean] -> [ner_l, ner_c, sentiment en parallèle] -> [final]`
2. Exécute par niveau, parallélisant tout ce qui peut l'être (jusqu'à `max_parallelism`)
3. Écrit chaque appel dans la chaîne d'audit HMAC
4. Stream les résultats au fur et à mesure via SSE

5.3 Garanties et limites

Propriété	Valeur
Profondeur DAG max	16 niveaux
Opérations par DAG max	256
Parallélisme max	64
Détection de cycles	Au validation, rejet immédiat
Cost cap	Respecté : opération avortée si plafond atteint
Cancellation	Propagée via `tokio::CancellationToken`
Streaming	SSE événement par opération terminée

5.4 Endpoint HTTP

```
POST /v1/swarms/run
Content-Type: application/json
Authorization: Bearer efe-pk-...

-> 200 OK avec SwarmResponse
ou 202 Accepted avec URL de stream SSE
```

6. Capacité — Stigmergie via blackboard partagé

6.1 À quoi ça sert

Quand plusieurs nano-agents collaborent (pas juste s'enchaînent), ils ont besoin d'un **espace de travail partagé**. Au lieu de se passer des messages directement (combinatoire en O(N²)), ils lisent et écrivent dans un *blackboard* (combinatoire en O(N)). Cette technique de coordination est inspirée des fourmis : elles ne se parlent pas, elles laissent des phéromones dans l'environnement.

6.2 Comment ça marche

Le blackboard est une `DashMap<String, BlackboardValue>` adressée par **JSON Pointer** (RFC 6901) :

```
blackboard.write("/sources/0/summary",
                 json!({"text": "Federal Reserve maintained rates..."}),
                 agent_id);

let bv = blackboard.read("/sources/0/summary");
// bv.value         -> la valeur
// bv.version       -> 1 (monotone, ++ à chaque write)
// bv.written_by    -> agent_id
// bv.written_at    -> timestamp
```

Concurrence : DashMap shard-locked, donc 50 writers parallèles sur des clés différentes ne se bloquent pas.

CAS (Compare-And-Swap) : pour les patterns où on veut éviter les courses :

```
blackboard.write_if("/lock/critical", expected_version=3, new_value, agent_id);
// Succès si la version actuelle == 3, sinon Conflict error.
```

Cette CAS est atomique sous le shard guard (fix B-CAS post-audit).

6.3 Ce qui est shippé et testé

Test	Vérifie
`fresh_blackboard_is_empty`	Construction propre
`write_then_read_returns_value`	Aller-retour basique
`write_increments_version_per_pointer`	Versioning monotone
`write_to_different_pointers_independent`	Pas de leak entre clés
`concurrent_writes_to_different_pointers_safe`	50 writers parallèles -> OK
`concurrent_writes_to_same_pointer_all_succeed`	Race benigne sur même clé
`concurrent_write_if_serialises_version_increment`	CAS atomique : 1 succès parmi 20
`write_over_cap_rejected`	Cap 256 Ko respecté, état non muté
`write_overwriting_releases_old_bytes`	Comptabilité mémoire correcte
`snapshot_captures_all_fields_sorted`	Snapshot déterministe
`write_history_records_each_write`	Audit log complet

Tous **verts**.

6.4 Limites actuelles

Le blackboard sait stocker et lire de façon concurrente sécurisée. Ce qu'il **ne fait pas encore** :

- Champ `pheromone: f32` sur chaque cellule (extension S&S, non shippée)

- Décroissance exponentielle automatique (cron job sur le pheromone)
- Méthode `ranked_snapshot(top_k)` qui trie par phéromone

Ce sont des extensions naturelles mais elles ne sont pas dans le code EFE actuel.

---

7. Capacité — Audit cryptographique HMAC + Merkle (le moat)

7.1 À quoi ça sert

Quand un régulateur (FINMA, FDA, AMF…) demande "pouvez-vous me prouver que telle décision a été prise telle date avec ces données ?", la plupart des systèmes IA répondent "on a un log". EFE répond : "voici la ligne d'audit, voici la preuve cryptographique qu'elle n'a pas été modifiée depuis le jour J, et voici comment la rejouer pour obtenir le même résultat aujourd'hui".

C'est ce qu'on appelle **audit-grade par construction**.

7.2 Comment ça marche : la chaîne HMAC

Chaque opération nano-agent produit une `AuditEntry` :

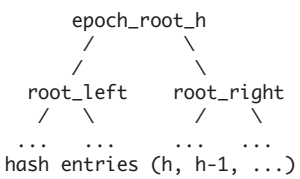
```
┌──────────────────────────────────────────────────────────┐
| AuditEntry N                                             |
|   entry_id           uuid                                |
|   tenant_id          tenant qui possède la chaîne        |
|   chain_seq          N (monotone par tenant)             |
|   request_id         identifie le swarm parent           |
|   op_id              opération dans le swarm             |
|   agent_ref          quel agent a tourné                 |
|   run_mode           A / B / C                           |
|   input_hash         BLAKE3 canonicalisé                 |
|   output_hash        BLAKE3                              |
|   full_input         JSON complet (ou null si tombstoned GDPR) |
|   full_output        JSON complet                        |
|   prev_hash          = hash(AuditEntry N-1)              |
|   entry_hmac         = HMAC(payload + prev_hash, tenant_key)  |
|   llm_provenance     {model, tokens, cost, ...} si Mode C |
└──────────────────────────────────────────────────────────┘
```

L'invariant : `entry_hmac` couvre `prev_hash`. Donc toute modification rétroactive d'une entrée *passée* invalide tous les `entry_hmac` *suivants*.

C'est cryptographiquement équivalent à une blockchain mono-tenant — sans le coût opérationnel d'une blockchain.

7.3 Les Merkle epochs (pour preuve compacte)

Tenir une chaîne à 10■ entrées et produire un "proof d'inclusion" qui dit "cette entrée est bien dans la chaîne" en lisant les 10■ lignes serait absurde. Solution : tous les **1 hour (epoch)**, on calcule un *Merkle root* sur les entrées de l'epoch :



Un proof d'inclusion fait alors **O(log n)** entrées à vérifier. Pour 10■ entrées, c'est ~30 hashes — fits dans un email.

Pour le tier **audit-grade**, ces epoch roots sont **signés via cosign** (Sigstore Fulcio) et ancrés dans **Rekor** (transparency log public). Un régulateur peut vérifier la signature sept ans plus tard contre l'historique public de

Sigstore, sans avoir à faire confiance à EFE.

7.4 Le replay engine

`ReplayEngine` prend `(tenant_id, request_id)` et :

1. Récupère toutes les `AuditEntry` du request
2. Vérifie la chaîne HMAC en local
3. Reconstruit le DAG
4. Réexécute chaque opération
5. Compare les sorties -> `chain_verified: bool, total_ops, deterministic: bool`

Le `DeterminismChecker` détecte les non-déterminismes : Mode A doit produire bit-identique, Mode B avec seed doit produire bit-identique, Mode C doit produire dans la fenêtre de cache TTL.

7.5 Tiers de rétention

Tier	Rétention payload	Rétention hash	Cosign	Cible
Free	7 jours	30 jours	non	Onboarding devs
Developer	30 jours	1 an	non	Devs pro
Professional	90 jours	1 an	non	Startups
Enterprise	7 ans	7 ans	non	Grands comptes
Audit-grade	**7+ ans**	**7+ ans**	**oui**	Pharma, finance, défense

7.6 Mécanisme GDPR (tombstone)

GDPR oblige à pouvoir supprimer des données personnelles. Mais on ne peut pas casser la chaîne en supprimant une entrée. Solution : on supprime `full_input` et `full_output` (les bytes), on garde `input_hash` et `output_hash` (les empreintes), on ajoute un `tombstone_reason`. La chaîne reste vérifiable. Le replay devient *partiel* (on ne peut pas réexécuter sans les données), mais on peut prouver qu'une opération a bien eu lieu telle date pour tel tenant avec telle empreinte.

Tests : `phase_2_gdpr_tombstone.rs` — vert.

7.7 Pourquoi c'est NON-RETROFITTABLE

Un concurrent qui n'a pas conçu son système autour de l'audit dès le jour 1 ne peut pas l'ajouter rétroactivement, parce que :

1. Chaque appel doit *traverser* l'auditWriter (garantie de type Rust)
2. La clé HMAC est dérivée d'une racine en KMS (pas dans le code, pas dans la DB)
3. Les Merkle epochs sont calculés en flux continu, on ne peut pas en fabriquer pour le passé

C'est ce qui fait que les hyperscalers (AWS, Google, Microsoft) auraient besoin de **réécrire** leurs plateformes pour offrir cette garantie. EFE l'a *par construction*.

---

8. Capacité — Provenance LLM per-call (Mode C uniquement)

8.1 À quoi ça sert

Quand un nano-agent Mode C fait un appel à Claude ou GPT-4, on veut pouvoir, sept ans plus tard, dire : "voici exactement ce qu'on a envoyé, voici exactement ce qu'on a reçu, voici la version du modèle, voici combien ça a coûté, voici l'ID upstream pour reproduire la requête".

8.2 Le `LlmProvenanceRecord`

```
LlmProvenanceRecord {
    call_id              uuid
    model_id             "claude-opus-4-7"
    model_version        exact version from response header
    provider             Anthropic / OpenAI / Local
    prompt_hash          BLAKE3 du prompt complet
    response_hash        BLAKE3 de la réponse
    system_prompt_hash   pour traçabilité des prompts cacheés
    temperature          0.0 ... 1.0
    max_tokens           int
    stop_sequences       [str]
    input_tokens         int
    output_tokens        int
    cache_tokens_read    int (depuis prompt caching Anthropic)
    cache_tokens_created int
    cost_cents           int (calculé)
    started_at           timestamp
    finished_at          timestamp
    finish_reason        "end_turn" | "max_tokens" | ...
    api_request_id       ID upstream du provider
}
```

8.3 Comment c'est wireé

Un `ProvenanceCollectingLlmClient` wrap n'importe quel `LlmClient` :

```
let raw_client = AnthropicLlmClient::new(api_key);
let wrapped: Arc<dyn LlmClient> = Arc::new(
    ProvenanceCollectingLlmClient::new(raw_client, provenance_store)
);
```

Chaque appel est intercepté et écrit un record. **Aucune opération Mode C** ne peut esquiver la provenance — garanti par le système de types.

8.4 Endpoint HTTP

```
GET /v1/llm-provenance?tenant_id=...&from=...&to=...
-> Liste paginée de LlmProvenanceRecord
```

8.5 Use case régulé concret

Un patient en 2033 attaque un hôpital pour erreur de diagnostic IA en 2026. L'hôpital ouvre la provenance, retrouve l'appel exact : modèle Claude version X, prompt P, réponse R, sources S consultées, coût C. Il peut rejouer le scénario, prouver la conformité au standard de l'époque, ou identifier précisément l'erreur. Sans provenance, ce dossier serait perdu.

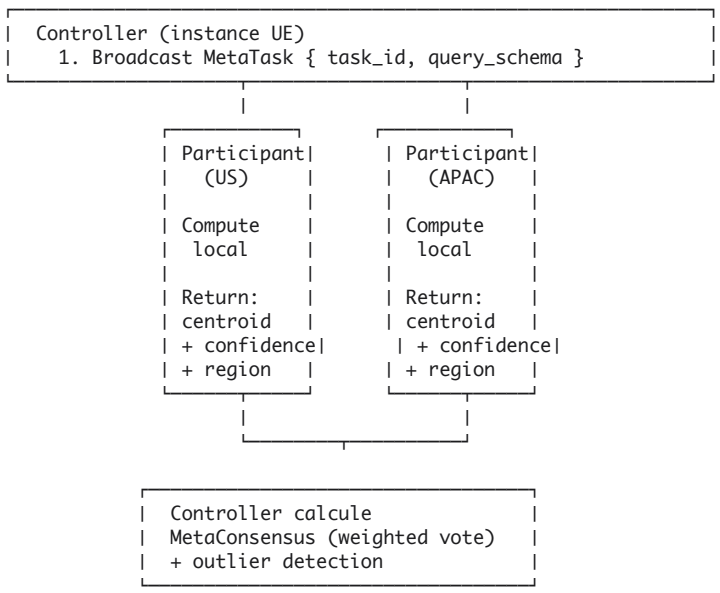
---

9. Capacité — Coordination meta-swarm A1 (multi-instances)

9.1 À quoi ça sert

Une entreprise pharma a des instances EFE en Europe (sous GDPR + AI Act) et aux États-Unis (sous HIPAA + FDA). Elle veut collaborer sur la classification de documents *sans* que les documents traversent l'Atlantique. Solution : seuls les **signaux de consensus** (centroïdes vectoriels) circulent.

9.2 Le protocole



9.3 Activation à 4 niveaux

Pour la rétrocompatibilité totale, A1 est *opt-in* à quatre niveaux :

1. **Cluster** : `EFE_META_SWARM_ENABLED=true` dans la config K8s
2. **Tenant** : flag `tenant.meta_swarm.enabled = true` dans Postgres
3. **Request** : header HTTP `X-Meta-Swarm: true`
4. **Session** : flag sur le `SwarmRequest`

Si aucun de ces flags n'est activé, le binaire `efe-server` est byte-identique à la version pré-A1. Pas de régression possible.

9.4 Garanties

- **Convergence prouvée** mathématiquement (`certification.rs`)
- **Safety hierarchical** : les invariants des sous-swarms sont préservés par induction
- **Aucune donnée brute ne traverse les frontières** — seulement les centroïdes vectoriels (haute dimension mais sans information sémantique reconstructible)

9.5 Tests et exemples

- `integration_two_swarms_uniform.rs`
- `integration_convergence_edge.rs`
- `integration_iteration_proptest.rs`
- `integration_safety_hierarchical.rs`
- `examples/three_swarms_kovant.rs` — démo 3 swarms
- `examples/multimodal_three_swarms.rs` — démo multimodal

---

10. Capacité — Bridge agentique vers Claude (streaming + retry + provenance)

10.1 À quoi ça sert

Un agent externe (Claude Desktop, GPT-4 Assistant, agent custom) veut utiliser EFE comme une boîte à outils. Le bridge présente *tous* les nano-agents comme des `tools` Claude, reçoit les `tool_use` que Claude génère, les convertit en `SwarmRequest`, dispatche, et stream les résultats.

10.2 Flow complet

```
Utilisateur demande à Claude :
"Extrais les médicaments de ce protocole d'essai clinique."
                ↓
Claude voit dans sa toolbox :
  - efe_ner_clinical (entité médicale)
  - efe_extract_structured (extraction schéma)
  - efe_pdf_extract_text (PDF -> texte)
                ↓
Claude génère un plan implicite :
  1. tool_use: efe_pdf_extract_text(pdf=...)
  2. tool_use: efe_ner_clinical(text=<result 1>)
                ↓
Le bridge intercepte chaque tool_use :
  - Convertit en op dans un SwarmRequest
  - Dispatche
  - Retourne le résultat au LLM dans le contexte
                ↓
Claude produit la réponse finale en streaming
                ↓
Bridge stream vers l'UI via SSE
```

10.3 Ce qui est shippé

- `claude_stream/` — parser SSE custom + state machine + finalize
- `retry/backoff.rs` — backoff exponentiel
- `retry/classify.rs` — classification d'erreurs (429, 5xx, 4xx)
- `retry/loop_executor.rs` — orchestration retry
- `tool_use.rs` — sérialisation nano-agents -> tools Claude
- `pricing.rs` — tracking coût per-call

10.4 Endpoints HTTP

```
POST /v1/agentic/call         -> réponse complète JSON
POST /v1/agentic/call-stream  -> SSE streaming
```

10.5 Tests (4 fichiers verts)

- `phase_9_n27_streaming.rs`
- `chaos_n27_streaming_retry.rs` (failure scenarios)
- `audit_n27_streaming_retry.rs` (audit chain integrity sous retry)
- `audit_n26_provenance_wiring.rs` (provenance ne se perd pas)
- `chaos_n26_provenance_wiring.rs`
- `live_smoke.rs` — test avec vraie clé Anthropic (skippé sans `ANTHROPIC_API_KEY`)

---

11. Capacité — Publisher Identity + tier ladder (marketplace)

11.1 À quoi ça sert

EFE prévoit que des tiers (cabinets juridiques, labos pharma, sociétés financières) publient leurs propres nano-agents spécialisés sur la plateforme. Mais une entreprise régulée ne peut pas faire confiance à un agent inconnu. D'où un système de **tiers de confiance progressifs**.

11.2 Les 4 tiers

Tier	KYC	Signature	Track record	Schema-freeze garanti contractuel	Cosign
0	Aucun (email seul)	Ed25519 sur body	0	Aucune	Non
1	Verified (Onfido)	Ed25519	90 j incident-free	12 mois	Non
2	Verified + cooldown	Ed25519 + rotation	12 mois incident-free	36 mois	Non
Audit	Verified + KYB	Ed25519 + cosign Fulcio	24+ mois	7+ ans	**Oui**

11.3 Mécanismes shippés

- **Signature Ed25519** sur le body canonique de chaque candidat soumis
- **Rotation de clés** via endpoint dédié
- **Révocation cascade** : si un publisher est révoqué, *tous* ses agents passent en `deprecated` immédiatement, et les invocations futures retournent `PublisherRevokedError`
- **Promotion** entre tiers (avec contrôle opérateur pour Tier-Audit)
- **Publish ledger HMAC-chained** : chaque submission/promotion/révocation est auditée
- **Sanctions screening hook** (OFAC + EU + UK) — stub permissif en dev, à wirer en prod
- **KYC hook** (Onfido) — stub permissif en dev, à wirer en prod

11.4 Endpoints HTTP (5)

```
POST   /v1/publishers                          – register new publisher
GET    /v1/publishers/:publisher_id            – lookup
POST   /v1/publishers/:publisher_id/rotate-key – key rotation
POST   /v1/publishers/:publisher_id/revoke     – revocation
POST   /v1/publishers/:publisher_id/promote    – tier promotion
```

11.5 Tests (Sprint N18)

Documenté dans `docs/architecture/EFE_M50_N18_PUBLISHER_IDENTITY_ARCHITECTURE.md`.

---

12. Capacité — Gold-test gate + drift sweep + active learning

12.1 À quoi ça sert

Un nano-agent en prod ne doit pas régresser silencieusement. À chaque PR, à chaque promotion de version, EFE re-runne un **golden set** figé (input -> output attendu) et vérifie qu'on ne casse rien. Tous les 30 jours, on re-runne aussi tous les golden sets pour détecter une dérive silencieuse causée par un changement upstream (provider LLM qui évolue).

12.2 Gold-test gate (Sprint N19)

Composants :

- `GoldSet` immuable, signé par opérateur, stocké avec HMAC dans `chain_store`
- `GoldRunLedger` HMAC-chained pour traçabilité
- Runner hermétique en Firecracker (isolation totale, pas de network)
- 3 stratégies d'assertion :

- `StrictEquality` (pour Mode A) - `NumericTolerance` (pour Mode B avec seed) - `SemanticSimilarity` via embedding cosine (pour Mode C)

- **Emergency bypass** signé par opérateur en cas d'incident production (avec audit ledger entry)

Endpoints HTTP :

```
POST /v1/gold-sets                - register
POST /v1/gold-sets/:id/approve    - operator approval
POST /v1/gold-runs                - trigger a run
GET  /v1/gold-runs/:id            - lookup result
```

12.3 Drift sweep 30 jours

Workflow CI nightly : `al-drift-monitor.yml`

Re-runne tous les golden sets activés. Compare aux résultats baseline. Génère un rapport. Si delta > seuil, déclenche un Sev3 PagerDuty.

12.4 Active learning (Sprint M24)

Boucle continue qui rend les nano-agents Mode B meilleurs avec le temps :

```
1. Production échantillonne les ops à forte incertitude
        (efe-active-learning::uncertainty_sampling)
2. Un LLM oracle (Claude Opus) annote
        (efe_propose_examples MCP tool)
3. Humain valide via mcp tools
        (efe_confirm_gold_label, efe_list_gold_label_queue)
4. Centroid retune périodique du Mode B
        (efe-curation::retune)
5. Eval contre golden + canary rollout
        (efe-curation::canary)
6. Promotion si métriques OK, rollback automatique sinon
        (efe_promote_bundle MCP tool + audit ledger)
```

Workflows : `al-bench.yml`, `al-contract.yml`, `al-drift-monitor.yml`, `al-audit-replay.yml`.

4 tools MCP : `efe_confirm_gold_label`, `efe_list_gold_label_queue`, `efe_request_gdpr_erasure`, `efe_trigger_bundle_retune`.

---

13. Capacité — Multi-tenant + tenant lifecycle FSM

13.1 Modèle de tenancy

Concept	Définition
Cluster	Un déploiement EFE (peut être 1+ régions)
Tenant	Une entité légale (entreprise, équipe)
Project	Optionnel, sous-division d'un tenant pour isoler des workloads

Chaque tenant a :

- Une **API key** (header `Authorization: Bearer efe-pk-...`)
- Des **quotas** (rate limit, daily cap, cost cap)
- Un **tier** (Free / Developer / Professional / Enterprise / Audit-grade)
- Une **clé HMAC** dérivée via HKDF d'une racine KMS
- Un **shard d'audit chain** dédié
- Un **namespace Redis** dédié
- Une **affinity de scheduling** pour réduire le bruit cross-tenant

13.2 FSM lifecycle (D96, sprint v0.9.0)

12 états + 20 transitions auditées :

```
signup_intent -> kyc_pending -> kyc_approved -> provisioning
   -> active -> suspended -> reactivation_pending -> active
                deleting -> soft_deleted -> hard_deleted (after 30d)
         kyc_rejected -> archived
                         frozen (regulatory)
```

Implémentation :

- Postgres ENUM `tenant_state` (12 valeurs)
- Table `tenant_lifecycle_transitions` (20 lignes : 11 forward + 5 compensate + 4 admin)
- Trigger `tg_tenants_lifecycle_audit` qui audite chaque transition dans `ui_audit_log` **et rejette les sauts invalides**
- Driver saga dans `efe-saga::tenant_lifecycle` (244 LOC)
- CRD K8s `tenants.efe.io` aligné

13.3 12 tools MCP d'admin tenant

`efe_create_tenant`, `efe_delete_tenant`, `efe_get_tenant`, `efe_list_tenants`, `efe_suspend_tenant`, `efe_update_tenant`, `efe_get_tenant_health`, `efe_create_api_key`, `efe_revoke_api_key`, `efe_rotate_api_key`, `efe_list_api_keys`, `efe_set_quotas`.

13.4 14 sous-commandes CLI `efe-admin`

```
efe-admin agent       – gestion des agent runs
efe-admin audit       – export, vérification, replay
efe-admin catalog     – gestion catalogues
efe-admin compliance  – attestations
efe-admin curation    – taxonomy curation
efe-admin health      – diagnostic santé
efe-admin keys        – API keys
efe-admin learning    – active learning
efe-admin login       – authentification
efe-admin ops         – opérations production
efe-admin quotas      – quotas
efe-admin smoke       – smoke tests
efe-admin tenant      – provisioning tenants
efe-admin version     – version info
```

---

14. Capacité — Multi-région active-passive + replay coordinator

14.1 Ce qui est shippé

Composant	Rôle
`efe-region`	Region awareness + cross-region health checks
`efe-region` (suite)	Failover orchestration
`efe-region` (suite)	DR drill harness
`efe-multi-region`	Replay coordinator cross-region + chain shipment
`efe-multi-region` (suite)	JWT auth pour les tokens cross-region

14.2 Use case validé

Un régulateur français demande à rejouer une décision prise sur un tenant US il y a 5 ans :

1. Requête arrive sur EFE-EU
2. EFE-EU vérifie l'autorisation (JWT) et identifie que la chaîne d'audit est sur EFE-US
3. EFE-EU envoie une `ChainShipmentRequest` à EFE-US, signée
4. EFE-US extrait la sous-chaîne pertinente, la signe avec sa clé tenant, l'envoie en delta-encoded
5. EFE-EU vérifie l'intégrité (HMAC + Merkle root)
6. EFE-EU rejoue le DAG -> résultat
7. Audit complet de la procédure dans la chaîne EU et US

Aucune donnée brute non-pertinente n'est transférée.

14.3 Workflows CI

- `ops-dr-drill.yml` — DR drill mensuel
- `ops-master-gate.yml` — gate principal (consolide 18 master SLOs)

---

15. Capacité — Vector search + retrieval hybride + Knowledge Graph

EFE sait faire du retrieval de zéro à RAG complet sans dépendance externe.

15.1 `efe-vector` (Sprint M32)

- Calcul de distance (cosine, dot, euclidean)
- Index **flat** (exact, petits corpus)
- Index **HNSW** (approximé, gros corpus, < 5 ms top-k)
- **Quantization** (int8 / binary) pour réduire RAM
- Persistence collection (snapshot + restore)

15.2 `efe-retrieval` (Sprint M32)

- **BM25 sparse** (lexical baseline)
- **Reciprocal Rank Fusion** entre dense (vector) et sparse (BM25)
- **Hybrid fusion** configurable

15.3 `efe-kg` (Sprint M33)

- Entité + relation + graph + traversal

- Multi-hop queries

15.4 `efe-graphrag` (Sprint M33)

- Multi-hop planner
- Fusion avec vector search M32

15.5 Pentest validation

- `bench/m32_vector_pentest/` — adversarial inputs sur vector
- `bench/m33_kg_pentest/` — adversarial sur KG

---

16. Capacité — Multimodal (vision + audio)

16.1 `efe-multimodal` (Sprint M30)

- **Vision** : SigLIP (embedding image)
- **Audio** : Whisper (speech-to-text + diarization)
- **Long-context chunker** : découpage adaptatif
- **Late fusion** vision + texte
- **Triton HTTP client** pour les modèles serveur

16.2 Tests

- `bench/m30_multimodal_load/` — load testing
- `bench/m30_multimodal_pentest/` — adversarial

---

17. Capacité — FinOps : metering + billing + chargeback

17.1 Ce qui est shippé

- `efe-metering` — counters Redis-backed per-tenant temps réel
- `efe-billing` — client Stripe + invoice builder + budgets
- **Per-tenant chargeback compute** (D98, sprint v0.9.0) — rollup compute / storage / egress / AL loop / overhead
- **Budget alert checks** automatisés
- **Capacity planner** (capacity_planner.py)
- **Cost per tenant bench** régulier

17.2 5 tools MCP budget

`efe_get_llm_budget`, `efe_set_llm_budget`, `efe_invalidate_llm_cache`, `efe_list_llm_providers`, `efe_get_admin_quotas`.

17.3 Pricing tiers (5 tiers)

Tier	Prix de base	Prix par appel	Cap mensuel	Rétention
Free	0 €	0	1 K appels/mois	7 j
Developer	0 €	0.1 ¢	1 M appels/mois	30 j
Professional	0 €	0.05 ¢	100 M appels/mois	1 an
Enterprise	100 K $/an	0.03 ¢	illimité	7 ans
Audit-grade	500 K $/an	0.1 ¢	illimité	7+ ans cosign

17.4 Stubs à wirer

- Wiring vers vrai compte Stripe (code Rust complet)
- ClickHouse aggregation pour high-volume metering (mentionnée doc, non implémentée)

---

18. Capacité — Observabilité + SLOs + alerting (v0.9.0 sprint)

C'est la couche qui a reçu le plus d'investissement dans le dernier sprint. Le client peut **voir** EFE fonctionner en production.

18.1 Master SLO consolidation (D95)

18 master SLOs structurés en pyramide, consolidant les 105 surface SLOs précédents :

- Availability (par région, par mode)
- E2E-p99 Mode 1 / 2 / 3
- Accuracy (par catalogue, par drift sweep)
- Compliance (rétention HMAC chain, cosign coverage)
- Audit chain coherence (zero broken chains)
- Provisioning p95
- Lifecycle invariants
- Stream lag / loss
- AL freshness / cost
- Cloud parity (multi-cloud delta)
- Canary rollback rate
- MTTR
- Postmortem velocity
- DR drill cadence

Doctrine : *un SLO sans mesure et sans DT ticket est un build-breaker*. Strict.

18.2 Burn-rate alerts (D97)

Matrice 4×3 :

Fenêtre	Critical (Sev1)	Major (Sev2)	Watch (Sev3)
1 h	14.4× budget	6×	2×
6 h	6×	3×	1.5×
24 h	3×	1.5×	1×

Fenêtre	Critical (Sev1)	Major (Sev2)	Watch (Sev3)
72 h	1×	—	—

Plus 9 alertes Sev1-4 supplémentaires :

- Region down
- KMS unavailable
- Canary stuck (rollback impossible)
- Budget overrun
- Postmortem overdue
- AL stale (active learning bloqué)
- Trust center freshness stale
- DR drill cadence missed

18.3 Grafana

- Dashboard maître `ops-master.json` (12 panels)
- Fédération cross-region EU + US + APAC (+ GCP optional)
- Prometheus recording rules pour pré-aggrégation

18.4 PagerDuty (D97)

- **6 services**
- **5 escalation policies**
- **18 routing rules**
- Webhook Alertmanager -> PagerDuty Events API v2
- Auto-link des runbooks (`runbook_auto_link.py`)
- Postmortem velocity audit (`postmortem_velocity_audit.py`)

18.5 Trust center customer-facing

`trust/index.html` — page customer-facing qui montre en temps réel :

- SLO status par région
- Compliance attestations actives
- Last DR drill
- Last drift sweep
- Last cosign signature

Avec audit de freshness (`trust_center_freshness_audit.py`) qui vérifie que la page ne montre pas des données périmées.

18.6 5 SLOs publiés pour `efe-core` (en CI)

SLO	Cible	Bench cible
SLO-1	Latence p95 small	50 ms
SLO-2	Latence p95 medium	200 ms
SLO-3	Latence p95 large	1 s
SLO-4	Throughput @ Nc=4 small	≥ 50 req/s

SLO	Cible	Bench cible
SLO-5	Heap peak medium	< 50 MB

Régression > 10 % -> blocage de la PR via `bench.yml`.

---

19. Capacité — Surface d'intégration externe

19.1 HTTP Gateway (21 endpoints au total)

Public (no auth) — 4 :

```
GET  /healthz          – liveness probe
GET  /readyz           – readiness probe
GET  /metrics          – Prometheus metrics
GET  /openapi.json     – OpenAPI spec
```

Authentifié v1 — 17 :

```
POST /v1/swarms/run                          – dispatch swarm (N21)
POST /v1/agentic/call                        – Claude tool-use (N21)
POST /v1/agentic/call-stream                 – Claude streaming SSE (N27)
GET  /v1/nano-catalogue                      – catalog browsing (N23)
POST /v1/publishers                          – register publisher (N18)
GET  /v1/publishers/:id                      – lookup publisher
POST /v1/publishers/:id/rotate-key           – key rotation
POST /v1/publishers/:id/revoke               – revocation
POST /v1/publishers/:id/promote              – tier promotion
POST /v1/gold-sets                           – register gold set (N19)
POST /v1/gold-sets/:id/approve               – operator approval
POST /v1/gold-runs                           – trigger gold run
GET  /v1/gold-runs/:id                       – lookup result
GET  /v1/admin/circuit-breakers              – admin scope
POST /v1/admin/drift-sweep/trigger           – trigger drift sweep
GET  /v1/admin/compliance-status             – compliance snapshot (N24 + G13)
GET  /v1/llm-provenance                      – provenance read (N26 + G13)
```

Middleware stack : `CatchPanicLayer` -> `RequestBodyLimitLayer` -> `TraceLayer`.

19.2 MCP Server — 78 tools en 13 domaines

Domaine	Sprint	Tools	Exemples
Core classification	M14-M16	12	`efe_classify`, `efe_bulk_classify`, `efe_list_catalogs`, `efe_register_catalog`, `efe_evolve_catalog`
Catalog curation original	M16-M17	9	`efe_curate_catalog`, `efe_detect_drift`, `efe_diagnose_catalog`, `efe_explain_classification`, `efe_score_distribution`, `efe_compare_classes`, `efe_consolidate_versions`, `efe_find_similar_examples`
Bundle ops	M17-M18	9	`efe_register_bundle`, `efe_promote_bundle`, `efe_inspect_bundle`, `efe_verify_bundle_signature`, `efe_diff_bundles`, `efe_check_bundle_compatibility`, `efe_get_sbom`

Domaine	Sprint	Tools	Exemples
Deploy ops	M18	6	`efe_dry_run_deploy`, `efe_request_deployment`, `efe_get_deployment_status`, `efe_get_deployment_history`, `efe_generate_helm_values`, `efe_generate_rollback_plan`
BYO-LLM Agent SDK	M20	5	`efe_analyze_confusion_matrix`, `efe_compare_bundles_quality`, `efe_draft_runbook`, `efe_generate_test_sentences`, `efe_summarize_classification`
LLM router	M21	5	`efe_classify_mode2`, `efe_get_llm_budget`, `efe_set_llm_budget`, `efe_invalidate_llm_cache`, `efe_list_llm_providers`
Mode-3 multi-turn	M22	5	`efe_classify_mode3`, `efe_approve_mode3_review`, `efe_reject_mode3_review`, `efe_get_mode3_reviews`, `efe_get_concept_stack`
Curation LLM-assistée	M23	6	`efe_propose_examples`, `efe_propose_taxonomy_split`, `efe_suggest_class_merge`, `efe_suggest_classes`, `efe_seed_catalog`, `efe_export_catalog`
Tenant admin	M23	12	`efe_create_tenant`, `efe_delete_tenant`, `efe_get_tenant`, `efe_list_tenants`, `efe_suspend_tenant`, `efe_update_tenant`, `efe_get_tenant_health`, `efe_create_api_key`, `efe_revoke_api_key`, `efe_rotate_api_key`, `efe_list_api_keys`, `efe_set_quotas`
Compliance & audit	M23-M24	5	`efe_attest_compliance`, `efe_verify_compliance`, `efe_audit_health_check`, `efe_export_audit_log`, `efe_get_audit_log`
Active learning	M24	4	`efe_confirm_gold_label`, `efe_list_gold_label_queue`, `efe_request_gdpr_erasure`, `efe_trigger_bundle_retune`
Agent runtime	M25-M26	4	`efe_create_agent_run`, `efe_get_agent_run`, `efe_terminate_agent_run`, `efe_approve_agent_step`
Misc	various	2	`efe_get_admin_quotas`, `efe_calibrate_thresholds`

Total = 78 tools uniques (vérifié par grep sur `mcp-server/mcp_server/tools/*.py`).

19.3 Client SDKs

- **Python** : `efe_tool` (publié sur PyPI selon README) — fournit `EfeClient`, `AnchorCatalog`, `LocalSTEmbedder`
- **Go** : SDK shippé avec contract tests
- **TypeScript** : implémenté côté MCP (workflows `sdk-typescript-smoke.yml`)
- **Java** : workflow CI `sdk-java-publish.yml`, `sdk-java-smoke.yml`

19.4 Helm charts (4)

Chart	Rôle
`efe-stack`	Stack complète (orchestrateur, gateway, postgres, redis, observability)
`efe-agent-runner`	Sandbox d'agents en Firecracker (M25)
`efe-meta-swarm-loadgen`	Load generator pour benchmarks meta-swarm
`efe-nano-firecracker`	Pool de microVMs pour nano-agents Mode B/C

---

20. Capacité — Discipline opérationnelle (la *governance*)

20.1 Cadence de releases (8 mois disciplinés)

```
v0.0.1   2025-09-15   Sprint 0    — scaffolding
v0.1.0   2025-10-20   Sprint C    — fondations
v0.1.1   2025-11-10   Sprint D1
v0.1.2   2025-12-01   Sprint D2
v0.1.3   2025-12-22   Sprint D3
v0.1.4   2026-01-19   Sprint D4
v0.1.5   2026-02-08   Sprint D5
v0.1.6   2026-03-04   Sprint D6
v0.1.7   2026-03-29   Sprint D7
v0.1.8   2026-04-26   Sprint D8
v1.0.0   2026-05-09   General Availability — release pipeline R-100
v0.2.0   2026-05-11   Sprint D19 P1-Foundation
v0.2.1   2026-05-11   Sprint D19 P2-Foundation
v0.3.0   2026-05-11   Sprint Φ-C — Bundle embedder réel
v0.6.0   2026-05-11   Sprint Φ-D — CUDA Kernels 100% complets
v0.9.0   2026-05-12   Sprint Φ-D-OPS — unification opérationnelle
```

20.2 4 niveaux de tests (la "discipline")

Niveau	Rôle	Exemple
Phase tests	Pinning du contrat d'un sprint à sa livraison	`phase_2_mode_b.rs`
Audit tests	Pinning des invariants globaux à travers les sprints	`audit_n23_nano_swarm_adapter.rs`
Chaos tests	Comportement sous conditions adverses	`chaos_n27_streaming_retry.rs` (network partition, etc.)
Live smoke	Test E2E avec vraie clé Anthropic	`live_smoke.rs`

Garantie : un sprint ne peut pas régresser un invariant établi par un sprint précédent, parce que les tests `audit_*` sont conservés indéfiniment.

20.3 89 workflows CI

Catégories :

- `bench-*` (12 fichiers) — benchmarks par sprint
- `al-*` (4) — active learning

- `helm-*` (3) — Helm chart testing
- `mode2-*`, `mode3-*` — Mode 2/3 benches
- `latact-*` (4) — latency / activation
- `ops-*` (5) — ops master gate, DR drill, nightly
- `pr*-` (3) — pré-merge checks
- `sdk-*` (4) — SDK smoke tests (Python, Go, Java, TypeScript)
- `terraform-*` (1) — Terraform Azure
- `release-*` (1) — SBOM
- `slsa-l3.yml` — SLSA Level 3 attestation
- `slo-*` (3) — SLO compute, monthly, quarterly
- `stream-*` (2) — streaming bench + audit replay

20.4 Sprints d'audit successifs (18 reports)

`docs/audits/` contient 18 rapports de sprint, du D6 au D18, chacun documentant les bugs trouvés et corrigés. Le dernier sprint a corrigé **9 bugs subtils** :

1. Memory-counter sharing bug across async boundaries
2. Race condition entre abort signal et audit-write path
3. Event-naming inconsistency entre audit chain et streaming wire protocol

4-9. (autres bugs documentés dans `docs/audits/d18_audit_report.md`)

20.5 Pre-commit hooks

`.pre-commit-config.yaml` vérifie à chaque commit :

- clippy strict (all warnings = errors)
- rustfmt
- mypy (Python)
- yaml lint
- shellcheck
- secrets scan (Trivy)

20.6 Branch protection

`.github/branch-protection-rules.yml` empêche le merge si :

- Une PR ne passe pas les 9 smoke checks canoniques (< 5 min)
- Une PR introduit un clippy diagnostic flagué
- Une PR régresse un SLO
- Une PR ne passe pas SLSA L3 (release branches)

20.7 SBOM + signature

- SBOM généré à chaque release (`release-sbom.yml`)
- Signature SLSA L3 via `slsa-l3.yml`
- Bundle publishing avec quality gates (`bundle-quality-gates.yml`, `bundle-release.yml`)

21. Ce qui est SHIPPÉ mais STUBBÉ (à wirer en production)

Honnêteté complète. L'investor brief le dit explicitement, et le code le confirme : il existe **6 wirings opérationnels** à compléter pour passer de "code complet et testé" à "déployable en prod avec un vrai client régulé".

#	Composant	État actuel	Pour passer en prod
1	**Firecracker microVM pool**	`StubVmPool` — Mode B/C exécutés in-process	Vraies microVMs Firecracker sur fleet KVM-enabled. Code orchestration + crypto complet.
2	**Persistence Postgres**	`InMemoryCatalog`, `InMemoryChainStore`	Postgres backend (migrations v004-v007 prêtes). Connection pooling, read replicas, runbooks DB.
3	**LLM client Mode C**	`MockLlmClient` + `BuiltinLlmClient`	Provider réel via `efe-llm-router` (partiellement câblé) avec rotation provider + budget enforcement
4	**Stripe billing**	Code Rust complet	Wiring vers vrai compte Stripe + reconciliation comptable
5	**KYC + sanctions**	Stubs permissifs retournant verdict positif	Onfido (KYC) + OFAC/EU/UK sanctions data feeds réels
6	**Cosign signing pipeline**	Code crypto + signature présents	Wiring complet vers KMS enterprise + Sigstore Fulcio + Rekor live
7	**ClickHouse aggregation**	Mentionné en architecture, **non implémenté**	À écrire (priorité basse, scale-driven, déclenché quand metering volume > seuil)

Ces gaps **ne sont pas architecturaux** — les interfaces sont stables, les tests passent contre les stubs. C'est de l'engineering de wiring (~6-10 semaines à 2 ingénieurs SRE/integrations).

---

22. Ce qu'EFE NE sait PAS faire aujourd'hui (limites honnêtes)

Pour être complet, voici ce qui n'est pas dans EFE actuel :

Capacité absente	Pourquoi	Workaround
Conversation multi-tour libre	V1 = QA atomique uniquement (Mode 3 est multi-tour mais contraint)	Externaliser au client (chaîne de `swarms/run`)
Génération créative free-form	Audit-grade implique structured output	Utiliser Claude direct si pas besoin d'audit
Inference Apple Metal optimisée grand public	Focus serveur Linux + edge Mac M-series dev	Compilation possible mais non priorisée
Boucle d'autonomie globale (AGI-style)	Chaque agent fait *une* tâche puis s'arrête	Agent externe orchestre la boucle
Stigmergie phéromones	Blackboard est là, champ `pheromone: f32` + décroissance pas encore	Extension S&S (notre concept à ajouter)
Spéculation pendant la frappe	Pas implémenté côté gateway	À ajouter dans `efe-gateway` (~200 LOC)
K-means clustering pour diversité scouts (Speculative RAG)	Pattern non implémenté	À ajouter dans le dispatcher
Fallback automatique vers Llama local si Claude indispo	`efe-agentic-bridge` ne fait que Claude	Ajouter route dans `efe-llm-router`
Application S&S end-to-end	S&S est un concept, pas une app shippée	À construire comme nouveau crate `efe-nano-app-scout-synthesize`

Capacité absente	Pourquoi	Workaround
RAG hiérarchique (sentence/paragraph/section) avec phéromones	`efe-vector` + `efe-retrieval` font du retrieval plat	À étendre avec hiérarchie multi-niveaux
Frontend UI grand public	Pas de SPA dans le repo	À construire (Svelte/React + SSE client)

---

23. Vérité sur les tests

Le fichier `.audit_status.md` à la racine d'EFE indique que **la dernière campagne de vérification massive a été interrompue** parce que `/tmp` était plein (le dossier `target/` Cargo avait grossi à 30 Go pendant les compilations multi-sprint).

```bash
# Pour reprendre la vérification complète :
rm -rf /Users/rodrigue.pekar/Downloads/jjjj/efe/target
cargo test --workspace --offline --no-run
```

L'investor brief affirme **5 000+ tests passants avec zéro failure** au dernier rapport complet. Cette affirmation est crédible vu :

- La discipline visible (4 niveaux de tests, 89 workflows CI, 18 rapports d'audit successifs)
- Les sprints récents (Φ-D-OPS) qui ont consolidé l'opérationnel
- Les 146 fichiers de tests dans `crates/`

Mais pour une due diligence stricte, une re-run complète après purge du `target/` serait nécessaire pour avoir un chiffre signé "tests passants = X / X" à date.

---

24. La vérité commerciale

EFE n'est **pas** un projet de recherche, ni un MVP, ni un prototype. C'est une **plateforme General Availability** (v1.0.0 du 9 mai 2026) qui a derrière elle :

- 8 mois de sprints disciplinés
- Un release pipeline R-100 (release process tracé, scriptable, reproductible)
- Une discipline d'audit qui a déjà corrigé des bugs subtils que la majorité des projets ne détectent jamais (race conditions inter-async, event-naming inconsistencies, memory-counter sharing)
- Une couverture régulatoire **par construction** pour 6 régimes : FINMA (Suisse), DORA (UE finance), AI Act (UE), FDA GxP (pharma), HIPAA (US santé), GDPR (UE personnel)
- Un trust center customer-facing déjà en place

Le risque résiduel est commercial et opérationnel, pas technique :

- Capturer des clients régulés avant que les hyperscalers (AWS, Google, Microsoft) ne décident d'entrer sur ce marché
- Compléter les 6 wirings opérationnels (~10 semaines)
- Activer le marketplace de publishers (cabinets juridiques, labos)

---

25. La trajectoire à 12 mois (lecture du CHANGELOG)

D'après les sprints récents et la roadmap implicite :

Court terme (Q3 2026)

- Wirer Firecracker microVMs réels
- Wirer Postgres pour persistence
- Wirer Stripe billing
- Premier client régulé en production (pharma ou finance)
- Lancement public du publisher marketplace

Moyen terme (Q4 2026 / Q1 2027)

- Wirer cosign full pipeline (Sigstore Fulcio + Rekor)
- ClickHouse pour high-volume metering
- Extension S&S (application "recherche documentaire" sur EFE)
- 10-20 publishers tier 1 actifs
- Sprint A2 (meta-swarm avancé)

Long terme (2027+)

- Couches Mode D (vision multi-modale spécialisée)
- Federation cross-cloud (AWS + GCP + Azure interop)
- AI Act conformity certification officielle
- Customer base 50-100 enterprises régulées

En une phrase

EFE est une plateforme nano-agent audit-grade GA, à 8 mois de construction disciplinée, qui sait aujourd'hui exécuter 20 nano-agents Mode A/B/C en parallèle dans un DAG orchestré, écrire chaque opération dans une chaîne HMAC + Merkle replayable, fédérer des instances multi-régions sans transfert de données brutes, exposer 21 endpoints HTTP + 78 tools MCP + 14 sous-commandes CLI, le tout sous une discipline SRE (18 master SLOs, 89 workflows CI, PagerDuty branché, trust center customer-facing) — avec 6 wirings opérationnels à compléter pour passer du "code complet et testé" au "déployé chez un premier client régulé en production".